

Programming Assignment 2

File Retrieval Engine, Application Layering, Multithreading

Instructions:

- Assigned date: Monday January 29th 2024
- Due date: 11:59pm on Wednesday February 14th 2024
- Maximum points: 100
- This programming assignment must be done individually.
- Please post your questions on the D2L Programming Assignments topic.
- Only the report must be uploaded to D2L. The code must be uploaded to the GitHub repository created for you for this assignment.
- Late submission will be penalized at 20% per late day.

This programming assignment aims to teach you about distributed systems architectures and processes, more precisely about Application Layering and Multithreading. As part of this homework assignment, you will need to implement a simple File Retrieval Engine that can index the files specified in a dataset and can execute queries over the indexed files in order to return the most relevant files that match a user specified query and the folders that they reside in. The programs that you need to implement as part of this assignment are limited to C++ and Java, and to the standard libraries. You can use any computer to develop the solution for this assignment, but you need to use a Linux machine, that runs Ubuntu 22.04, from Chameleon Cloud to evaluate the performance of your solution.

1 File Retrieval Engine Architecture

The File Retrieval Engine must adhere to the architecture shown in Figure 1 and must contain three components: **AppInterface**, **ProcessingEngine** and **IndexStore**.

The **AppInterface** component is responsible with implementing a command line interface that the user can use to interact with the File Retrieval Engine, with interpreting indexing and search commands submitted by the user, with forwarding the commands to the **ProcessingEngine** and with printing the results of the commands on screen.

The **ProcessingEngine** component is responsible with running the indexing and search commands received from the **AppInterface** component. In terms of indexing, the **ProcessingEngine** must use multiple worker threads, that are configured at execution time, to build an index from the files found at a specified directory. Each worker thread is responsible with building a local index and with forwarding the local index to the

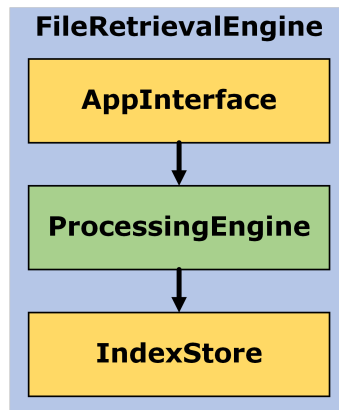


Figure 1: File Retrieval Engine Architecture.

IndexStore in order to ultimately build a global index. In terms of search, the **ProcessingEngine** must break down a search query into one or more single term queries, must retrieve for each single term query the list of files in which the term can be found and the number of occurrences, and must combine the list of files from each single term query result, sort the files by occurrence and return the top 10 files.

The **IndexStore** component is responsible with storing the global index and providing the means to query the global index. More specifically, it should expose an interface to update the global index with a local index and an interface to return the list of files and the number of occurrences in which a term can be found.

2 File Retrieval Engine Specification

The File Retrieval Engine must support the following commands:

- **quit:** this command closes the application by gracefully shutting down all of the active components (all worker threads running as part of the application).
- **index <dataset path>:** this command needs to tell the File Retrieval Engine to crawl and find all of the files in the given dataset path, and must build an index from those files. The File Retrieval Engine must use multiple threads to index the dataset. Each thread must build a local index and each thread is responsible with updating the global index with the local index at the end of the index build process. Sequences of alphanumeric characters (a-zA-Z0-9) that are separated by any other non-alphanumeric character(s) are considered as terms that need to be indexed.
- **search <AND query>:** this command needs to tell the File Retrieval Engine to process a search query. The search query is expressed as an AND query, that returns all of the files that contain all of the terms from the AND query. The returned files are to be sorted by the number of accumulated occurrences of all terms in each file, and only the top 10 files need to be printed to screen. For example, if the user inputs the following query: *cats AND dogs*, the File Retrieval Engine must return all of the files that contain *cats* and *dogs*, must sort the returned files by the total number of

occurrences of both *cats* and *dogs* in each file, and must return only the top 10 files.

3 Datasets

You need to evaluate the performance of your solution over the [Gutenberg Project Datasets](#). There are 5 datasets with increasing size and number of files. Each dataset contains a number of 16 folders each containing a collection of ASCII TXT documents representing books collected from the Internet. You may test your program with other datasets, but your program must run without errors over this dataset to gain full points.

4 Performance Evaluation

You need to evaluate the performance of your solution with increasing number of indexing worker threads (1, 2, 4, 8, 16 threads) for each dataset. For this you will need to submit in the report two plots and a discussion. The plots need to capture the performance of your solution in terms of indexing and search. For indexing you need to measure the performance of each dataset in terms of number of files indexed per second (y-axis) with increasing number of worker threads (x-axis) for each dataset (5 lines/curves). For search you need to create a bar plot where you showcase the average search latency and standard deviation of search queries over 100 randomly generated queries.

Finally, you must include in your report a discussion about the interpretation of your evaluation, answering the following questions:

- What data structure did you use to implement the index(es) and why?
- How did you make sure that multiple worker threads were able to create a global index in a safe and correct way?
- How does the dataset size influence indexing and search performance?
- Why does your solution exhibit the performance that you measured?
- Is there anything that you could have done better?
- What were the challenges?

5 Chameleon Cloud Testbed

You need to use the Chameleon Testbed to evaluate the performance of your application. When the code is ready and working correctly, spawn an instance on Chameleon Cloud and evaluate your code over the 5 datasets on that instance. The Chameleon instance needs to be of one of these types:

- `compute_icelake_r650` or `compute_icelake_r750`
- `compute_cascadelake` or `compute_cascadelake_r`
- `compute_skylake`
- `compute_haswell`

6 Where you will submit

You will have to submit your report to D2L and the code to a private git repository created for you for this programming assignment. The repository is created through GitHub Classroom and you will need to accept the assignment before you can clone it, by accessing this invitation link: <https://classroom.github.com/a/hqMOT4I->. The first time you access the invitation link, the system will ask you to identify yourself. Please select the identifier that contains your school email address. After that you can clone the repository. Then you will have to add or update the source code and readme. Your submission on GitHub Classroom will not be graded unless you submit the report to D2L. The timestamp of your report on D2L will be used to determine if the submission is on time or late. If you cannot access your repository, please reach out at aorhean@depaul.edu.

7 What you will submit

As you work on the coding aspect of the assignment you should submit to the git repository each small functionality that you implement. When you have finished implementing the complete assignment you should submit your report to D2L. You need to hand in:

1. **Source code (50%):** All of the source code, including comments and a CMake build file for a C++ solution, or a Maven build file for a Java solution.
2. **README.md (10%):** A detailed manual describing the structure of your files and directory organization. The manual should contain a description of how to build the program and how to run the program.
3. **Report (40%):** A written PDF document describing the overall assignment completion, along with screenshots and text answers to the assignment questions. Make sure to label the answers with the appropriate part and question number.

Do not upload the datasets!

Submit step 1: code/build file/readme through git.

Submit step 2: report through D2L.

Grades for late submission will be lowered 20% per late day.